Functional (Style)

Quentin Crain

```
nums = [4, 2, 7, 22]
```

New list where each number in `nums` except 7, which is thrown out, is incremented by 3.

Before

```
nums = [4, 2, 7, 22]
```

After

```
nums = [4, 2, 7, 22]

new_nums = []

for num in nums:
    if num != 7:
        new_nums.append(num + 3)

return new_nums
```

Prefer functions!

Before

```
nums = [4, 2, 7, 22]

new_nums = []

for num in nums:
    if num != 7:
        new_nums.append(num + 3)

return new_nums
```

After

```
import operator
import functools

add_3 = functools.partial(operator.add, 3)
ne_7 = functools.partial(operator.ne, 7)

nums = [4, 2, 7, 22]

new_nums = []

for num in nums:
    if ne_7(num):
        new_nums.append(add_3(num))

return new_nums
```

Using curry (partial application) allowing for reuse & refactoring.

Prefer list comprehension!

Before

```
import operator
import functools

add_3 = functools.partial(operator.add, 3)
ne_7 = functools.partial(operator.ne, 7)

nums = [1, 2, 7, 22]

new_nums = []

for num in nums:
    if ne_7(num):
        new_nums.append(add_3(num))

return new_nums
```

After

```
import operator
import functools

add_3 = functools.partial(operator.add, 3)
ne_7 = functools.partial(operator.ne, 7)

nums = [1, 2, 7, 22]

!new_nums = [add_3(num) for num in nums if ne_7(num)]

return new_nums
```

This is probably where we stop with Python, but ..

Prefer functions!

Before

```
import operator
import functools

add_3 = functools.partial(operator.add, 3)
ne_7 = functools.partial(operator.ne, 7)

nums = [4, 2, 7, 22]

new_nums = [add_3(num) for num in nums if ne_7(num)]

return new_nums
```

After

```
import operator
import functools

add_3 = functools.partial(operator.add, 3)
ne_7 = functools.partial(operator.ne, 7)

nums = [4, 2, 7, 22]

!nums_without_7 = filter(ne_7, nums)

!new_nums = map(add_3, nums_without_7)

return new_nums
```

Prefer functions!

Before

```
import operator
import functools

add_3 = functools.partial(operator.add, 3)
ne_7 = functools.partial(operator.ne, 7)

nums = [4, 2, 7, 22]

nums_without_7 = filter(ne_7, nums)

new_nums = map(add_3, nums_without_7)

return new_nums
```

After

```
import operator
import functools

add_3 = functools.partial(operator.add, 3)
ne_7 = functools.partial(operator.ne, 7)

!filter_ne_7 = functools.partial(filter, ne_7)
!map_add_3 = functools.partial(map, add_3)
!filter_ne_7_then_map_add_3 = lambda ns: map_add_3(filter_ne_7(ns))

nums = [4, 2, 7, 22]

!new_nums = filter_ne_7_then_map_add_3(nums)

return new_nums
```

Explicit looping is gone, curry & compose, "higher-order" concepts of `map`, `filter`.

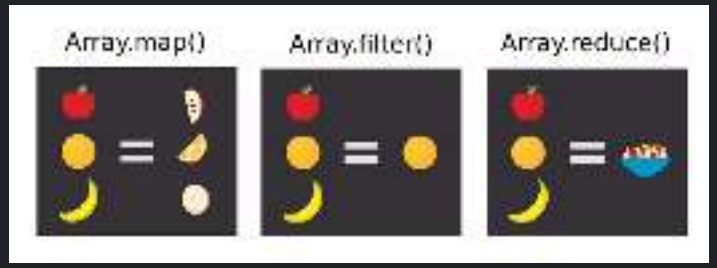
Before

```
nums = [4, 2, 7, 22]

new_nums = []

for num in nums:
    if num != 7:
        new_nums.append(num + 3)

return new_nums
```

After

```
import operator
import functools

add_3 = functools.partial(operator.add, 3)
ne_7 = functools.partial(operator.ne, 7)

filter_ne_7 = functools.partial(filter, ne_7)
map_add_3 = functools.partial(map, add_3)
filter_ne_7_then_map_add_3 = lambda ns: map_add_3(filter_ne_7(ns))

nums = [4, 2, 7, 22]

new_nums = filter_ne_7_then_map_add_3(nums)

return new_nums
```

`map`, `filter`, `reduce`



Curry (`partial`)

https://en.wikipedia.org/wiki/Currying

(Function) Composition

https://en.wikipedia.org/wiki/Function_composition_(computer_science)

COMMENTS

QUESTIONS

CONCERNS

Functional is better!

Elixir

```
iex> nums = [4, 2, 7, 22]
[4, 2, 7, 22]

iex> nums
     |> Enum.filter(fn n -> n != 7 end)
     |> Enum.map(fn n -> n + 3 end)
[7, 5, 25]

# or, with the capture shorthand:
iex> nums
     |> Enum.filter(& &1 != 7)
     |> Enum.map(& &1 + 3)
[7, 5, 25]
```

Racket

```
> (let [(nums '(4 2 7 22))
        (add-3 (curry + 3))
        (ne-7 (compose not (curry = 7)))]
    (map add-3 (filter ne-7 nums))
    )
'(7 5 25)

; absurd, or is it?!?
> (let* [(nums '(4 2 7 22))
         (add-3 (curry + 3))
         (map-add-3 (curry map add-3))
         (ne-7 (compose not (curry = 7)))
         (filter-ne-7 (curry filter ne-7))
         (filter-ne-7-then-map-add-3 (compose map-add-3 filter-ne-7))]
    (filter-ne-7-then-map-add-3 nums)
    )
'(7 5 25)
```